

Programación Orientado a Objetos (Herencia y Relaciones)

Natalia Gallego Barrero
Juan Carlos Mejía Cano
Docente: Alejandro Rodas Vásquez
Universidad Tecnológica de Pereira

12 de mayo de 2026

1. Introducción

El presente proyecto consiste en el desarrollo de un sistema de gestión agrícola implementado en Python bajo una arquitectura modular por componentes. El sistema permite registrar productos agrícolas, realizar compras, generar facturas y consultar el historial de compras de los clientes mediante su número de cédula.

Este sistema surge de la necesidad de organizar la información relacionada con la venta de productos agrícolas, permitiendo llevar un control sobre los productos disponibles, los clientes y las facturas generadas.

El sistema permite:

- Clases y objetos.
- Herencia.
- Composición.
- Encapsulamiento.
- Modularidad.
- Arquitectura por capas.

Además, se aplican conceptos de Programación Orientada a Objetos como herencia, encapsulamiento y composición.

2. Arquitectura del Proyecto

El proyecto fue desarrollado siguiendo una arquitectura por componentes con el objetivo de separar responsabilidades y mejorar la organización del código.

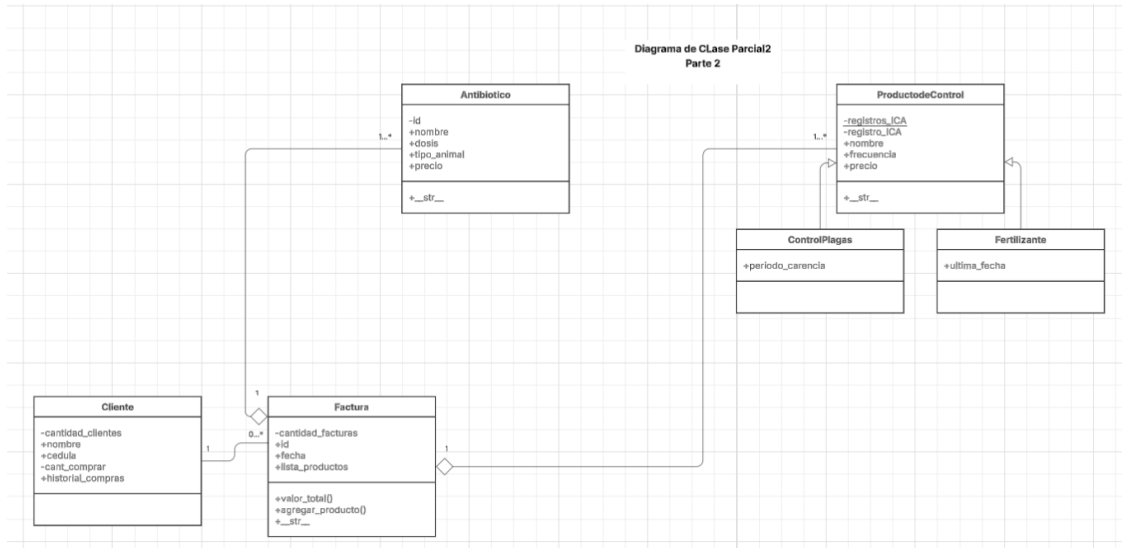


Diagrama de clases

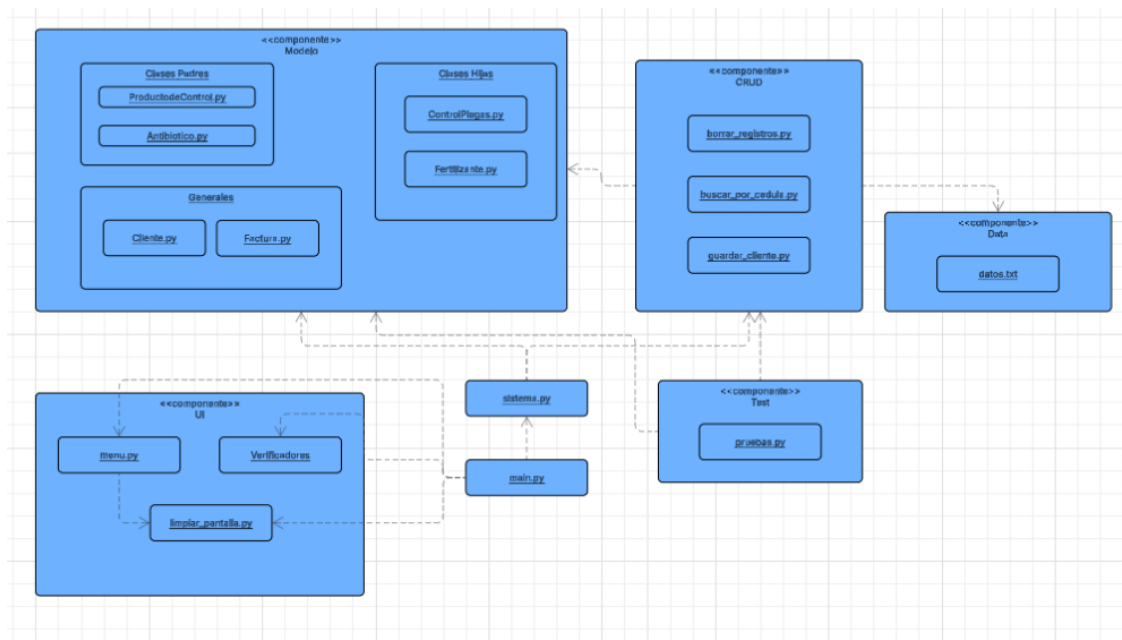


Diagrama de componentes

2.1. Componente Modelo

Contiene todas las clases principales del sistema.

- ProductoControl.py
- Antibiotico.py
- ControlPlagas.py
- Fertilizante.py
- Cliente.py
- Factura.py

2.2. Componente UI

Contiene toda la interacción con el usuario.

- menu_opciones.py
- limpiar_pantalla.py
- verificar_cadenas.py
- verificar_fecha.py
- verificar_valores_numericos.py

2.3. Componente CRUD

Gestiona el almacenamiento y recuperación de información.

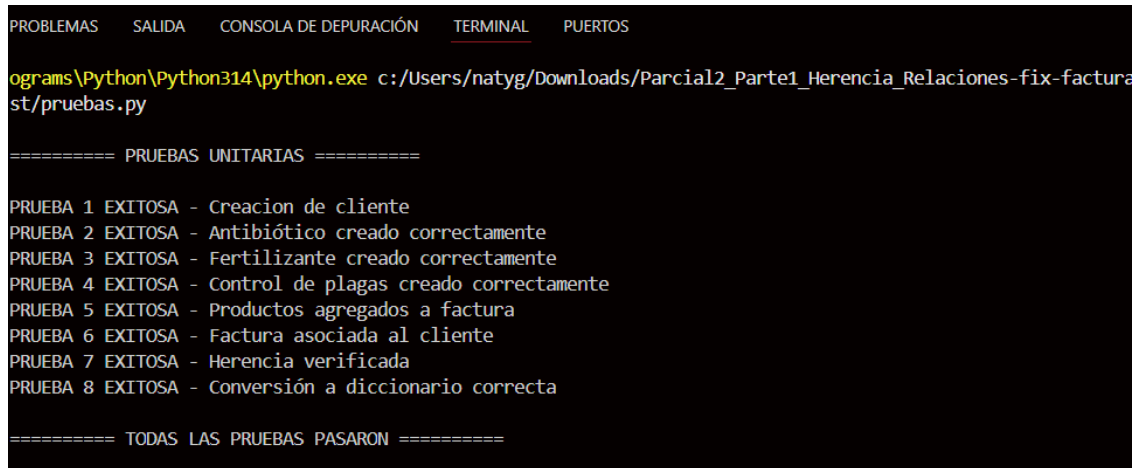
- guardar_cliente.py
- buscar_por_cedula.py
- borrar_registros.py

2.4. Componente Data

Almacena la persistencia de datos mediante el archivo:

- clientes.txt

2.5. Componente Test



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

ograms\Python\Python314\python.exe c:/Users/natyg/Downloads/Parcial2_Parte1_Herencia_Relaciones-fix-factura-st/pruebas.py

===== PRUEBAS UNITARIAS =====

PRUEBA 1 EXITOSA - Creacion de cliente
PRUEBA 2 EXITOSA - Antibiótico creado correctamente
PRUEBA 3 EXITOSA - Fertilizante creado correctamente
PRUEBA 4 EXITOSA - Control de plagas creado correctamente
PRUEBA 5 EXITOSA - Productos agregados a factura
PRUEBA 6 EXITOSA - Factura asociada al cliente
PRUEBA 7 EXITOSA - Herencia verificada
PRUEBA 8 EXITOSA - Conversión a diccionario correcta

===== TODAS LAS PRUEBAS PASARON =====
```

Ejecución de pruebas unitarias

Contiene las pruebas unitarias del sistema.

3. Herencia en el Proyecto

La herencia se implementó entre las clases:

- ProductoControl.
- ControlPlagas.
- Fertilizante.

La clase ProductoControl actúa como clase padre y contiene los atributos comunes.

Las clases hijas reutilizan dichos atributos y agregan características específicas.

3.1. Ventajas obtenidas

- Reutilización de código.
- Reducción de redundancia.
- Mejor organización.
- Mayor escalabilidad.

4. Composición en el Proyecto

La composición se observa principalmente entre:

- Cliente y Factura.
- Factura y Productos.

Cada cliente puede contener múltiples facturas dentro de su historial de compras.

Cada factura contiene múltiples productos.

5. Debug

```
✓ cliente = <Modelo.ClasesGenerales.Cliente.Cliente object at 0x000002552A9E4AD0>
> special variables
  cedula = 1088825719
✓ historial_compras = {1: <Modelo.ClasesGenerales.Factura.Factura object at 0x000...
  > special variables
  > function variables
  > 1 = <Modelo.ClasesGenerales.Factura.Factura object at 0x000002552A9E4980>
    len() = 1
    nombre = 'Natalia'
    _cant_comprar = 1
    _cantidad_clientes = 1
    dosis = 12
    frecuencia = 15
    ica = 1234
    idp = 2
    nombre = 'Natalia'
    op = 2
    opcion_control = 1
    opcion_principal = 3
    opcion_tipos_producto = 2
    periodo = 7
    precio = 40000
```

Composición cliente

```

▼ factura_actual = <Modelo.ClasesGenerales.Factura.Factura object at 0x000002552A...
  cliente = None
  > fecha = datetime.date(2026, 5, 12)
  id = 1
▼ lista_productos = [<Modelo.ClasesHijas.ControlPlagas.ControlPlagas object at 0...
  > special variables
  > function variables
▼ 0 = <Modelo.ClasesHijas.ControlPlagas.ControlPlagas object at 0x000002552A9E4...
  > special variables
  frecuencia = 15
  nombre = 'Mataplagas'
  periodo_carencia = 7
  precio = 20000
  _registroICA = 1234
▼ 1 = <Modelo.ClasesPadre.Antibiotico.Antibiotico object at 0x000002552A9E4830>
  > special variables
  dosis = 12
  nombre = 'Antibiotico'
  precio = 40000
  tipo_animal = 'porcino'
  _Antibiotico__registros_antibiotico = 1
  _id = 1
  len() = 2
  valor_total = 60000
  _contador_facturas = 1

```

Composición facturas

```

  ▾ productos = {'productos_control': {'plagas': {...}, 'fertilizantes': {}}, 'ant
    > special variables
    > function variables
  ▾ 'productos_control' = {'plagas': {1: <Modelo.ClasesHijas.ControlPlagas.Contro
    > special variables
    > function variables
  ▾ 'plagas' = {1: <Modelo.ClasesHijas.ControlPlagas.ControlPlagas object at 0x6
    > special variables
    > function variables
  ▾ 1 = <Modelo.ClasesHijas.ControlPlagas.ControlPlagas object at 0x000002552A9
    > special variables
      frecuencia = 15
      nombre = 'Mataplagas'
      periodo_carencia = 7
      precio = 20000
      _registroICA = 1234
    len() = 1
    > 'fertilizantes' = {}

```

Herencia

6. Conclusiones

A partir del desarrollo del sistema de facturación para la tienda agrícola, se pueden establecer las siguientes conclusiones:

- La implementación de la **herencia** permitió estructurar el sistema de manera jerárquica, facilitando la reutilización de código y evitando la duplicación de atributos en las clases hijas. Esto se evidenció en las clases de antibióticos y productos de control, donde se definieron atributos comunes en las clases padre y se especializaron en las clases hijas.
- La correcta definición de **atributos y métodos** en cada clase permitió representar de manera adecuada los elementos del mundo real, como clientes, productos y transacciones, logrando un sistema coherente y funcional.
- Finalmente, este proyecto permitió afianzar los conceptos de la programación orientada a objetos, demostrando su importancia en el desarrollo de sistemas estructurados, mantenibles y escalables.